ShrinkageTrees: Bayesian Tree Ensembles for Survival Analysis and Causal Inference

Tijn Jacobs

2026-03-03

Contents

0.1 Introduction . . . 1
0.2 The PDAC Dataset . . . 2
0.3 Single-Forest Models . . . 3
0.4 High-Dimensional Survival Analysis . . . 6
0.5 Causal Forest Models . . . 8
0.6 S3 Methods . . . 12
0.7 Multi-Chain MCMC . . . 19
0.8 Case Study: TCGA PAAD (Full Analysis) . . . 22
0.9 Session Info . . . 23
0.10 References . . . 24

0.1 Introduction

ShrinkageTrees is an R package for Bayesian regression tree ensembles designed for two challenging settings in modern statistics:

- **High-dimensional survival analysis** — right-censored outcomes with many more covariates than observations.
- **Causal inference with heterogeneous treatment effects** — estimating patient-level conditional average treatment effects (CATEs) and the population-level average treatment effect (ATE).

The package builds on the well-known **BART** framework (Chipman, George & McCulloch 2010) and extends it in three directions:

1. **Horseshoe leaf shrinkage.** A global–local horseshoe prior is placed directly on the step heights (leaf parameters) of each tree, allowing strong signals to retain their magnitude while noise is aggressively shrunk toward zero.
2. **AFT survival models.** Right-censored survival times are handled through an accelerated failure time (AFT) formulation with Bayesian data augmentation, enabling full posterior inference without proportional-hazards assumptions.
3. **BCF-style causal decomposition.** Following Hahn, Murray & Carvalho (2020), the outcome is decomposed into a prognostic function $\mu(\mathbf{x})$ and a treatment-effect function $\tau(\mathbf{x})$, each estimated by a separate tree ensemble.

0.1.1 Package map

Function	Task	Prior
`HorseTrees()`	Prediction (continuous / binary / survival)	Horseshoe
`ShrinkageTrees()`	Prediction — flexible prior choice	Horseshoe, DART, BART, ...
`SurvivalBART()`	Survival prediction	Classical BART
`SurvivalDART()`	Sparse survival prediction	DART (Dirichlet)
`SurvivalBCF()`	Causal survival inference	BCF (classical)
`SurvivalShrinkageBCF()`	Sparse causal survival inference	BCF + DART
`CausalHorseForest()`	Causal inference (all outcomes)	Horseshoe
`CausalShrinkageForest()`	Causal inference — flexible prior	Horseshoe, DART, BART, ...

All model-fitting functions return an S3 object with consistent `print()`, `summary()`, `predict()`, and `plot()` methods.

0.2 The PDAC Dataset

The package ships with `pdac`, a processed subset of the TCGA pancreatic ductal adenocarcinoma (PAAD) cohort. The dataset contains overall survival times, a binary treatment indicator (radiation therapy vs. control), clinical covariates, and expression values of approximately 3,000 genes selected by median absolute deviation.

```r
library(ShrinkageTrees)
data("pdac")

# Dimensions and column overview
cat("Patients:", nrow(pdac), "\n")
#> Patients: 130
cat("Columns :", ncol(pdac), "\n")
#> Columns : 3032
cat("Clinical columns:", paste(names(pdac)[1:13], collapse = ", "), "\n")
#> Clinical columns: time, status, treatment, age, sex, grade, tumor.cellularity, tumor.purity, absolut
cat("Survival: time (months), censoring rate =",
    round(1 - mean(pdac$status), 2), "\n")
#> Survival: time (months), censoring rate = 0.47
cat("Treatment: radiation =", sum(pdac$treatment),
    "/ control =", sum(1 - pdac$treatment), "\n")
#> Treatment: radiation = 36 / control = 94
```

We separate the outcome, treatment, and covariate matrix for the analyses below.

```r
time      <- pdac$time
status    <- pdac$status
treatment <- pdac$treatment
X         <- as.matrix(pdac[, !(names(pdac) %in% c("time", "status", "treatment"))])
```

0.3 Single-Forest Models

0.3.1 HorseTrees — binary outcome (propensity scores)

Before fitting causal models we estimate propensity scores $\hat{e}(\mathbf{x}) = P(W = 1 \mid \mathbf{x})$ using `HorseTrees()` with a binary outcome. The probit link is used internally: predictions are on the latent Gaussian scale and can be converted to probabilities with `pnorm()`.

The code block below uses reduced MCMC settings for illustration. A real analysis would use `N_post = 5000, N_burn = 5000`.

```
ps_fit <- HorseTrees(
  y            = treatment,
  X_train      = X,
  outcome_type = "binary",
  k            = 0.1,
  N_post       = 5000,
  N_burn       = 5000,
  verbose      = FALSE
)

propensity <- pnorm(ps_fit$train_predictions)
```

For the remainder of this vignette we use a short synthetic run to keep build time low.

```
set.seed(1)
n <- 80;  p <- 10
X_syn  <- matrix(rnorm(n * p), n, p)
W_syn  <- rbinom(n, 1, pnorm(0.8 * X_syn[, 1]))

ps_fit <- HorseTrees(
  y            = W_syn,
  X_train      = X_syn,
  outcome_type = "binary",
  number_of_trees = 5,
  k            = 0.5,
  N_post       = 50,
  N_burn       = 25,
  verbose      = FALSE
)

propensity_syn <- pnorm(ps_fit$train_predictions)
cat("Propensity scores - range: [",
    round(range(propensity_syn), 3), "]\n")
#> Propensity scores - range: [ 0.469 0.539 ]
```

0.3.2 HorseTrees — survival outcome

`HorseTrees()` handles right-censored data via an AFT model. Pass `outcome_type = "right-censored"` and provide the `status` vector.

```r
set.seed(2)
log_T <- X_syn[, 1] + rnorm(n)
C     <- rexp(n, 0.5)
y_syn   <- pmin(exp(log_T), C)
d_syn   <- as.integer(exp(log_T) <= C)

ht_surv <- HorseTrees(
  y               = y_syn,
  status          = d_syn,
  X_train         = X_syn,
  outcome_type    = "right-censored",
  timescale       = "time",
  number_of_trees = 5,
  N_post          = 50,
  N_burn          = 25,
  store_posterior_sample = TRUE,
  verbose         = FALSE
)

cat("Posterior mean log-time (first 5 obs):",
    round(ht_surv$train_predictions[1:5], 3), "\n")
#> Posterior mean log-time (first 5 obs): 1.204 1.319 1.354 1.389 1.247
cat("Posterior sigma - mean:",
    round(mean(ht_surv$sigma), 3), "\n")
#> Posterior sigma - mean: 0.981
```

0.3.3 ShrinkageTrees — flexible prior choice

`ShrinkageTrees()` generalises `HorseTrees()` by exposing the `prior_type` argument. Available priors are `"horseshoe"`, `"horseshoe_fw"`, `"horseshoe_EB"`, `"half-cauchy"`, `"standard"` (BART), and `"dirichlet"` (DART).

```r
set.seed(3)
y_cont <- X_syn[, 1] + 0.5 * X_syn[, 2] + rnorm(n)

# Horseshoe prior (default for HorseTrees)
fit_hs <- ShrinkageTrees(
  y               = y_cont,
  X_train         = X_syn,
  outcome_type    = "continuous",
  prior_type      = "horseshoe",
  local_hp        = 0.1 / sqrt(5),
  global_hp       = 0.1 / sqrt(5),
  number_of_trees = 5,
  N_post          = 50,
  N_burn          = 25,
  verbose         = FALSE
)

# Forest-wide horseshoe (horseshoe_fw)
```

```r
fit_fw <- ShrinkageTrees(
  y               = y_cont,
  X_train         = X_syn,
  outcome_type    = "continuous",
  prior_type      = "horseshoe_fw",
  local_hp        = 0.1 / sqrt(5),
  global_hp       = 0.1 / sqrt(5),
  number_of_trees = 5,
  N_post          = 50,
  N_burn          = 25,
  verbose         = FALSE
)

cat("Horseshoe   - train RMSE:",
    round(sqrt(mean((fit_hs$train_predictions - y_cont)^2)), 3), "\n")
#> Horseshoe   - train RMSE: 1.217
cat("Horseshoe FW- train RMSE:",
    round(sqrt(mean((fit_fw$train_predictions - y_cont)^2)), 3), "\n")
#> Horseshoe FW- train RMSE: 1.23
```

0.3.4 Survival wrappers: SurvivalBART and SurvivalDART

The wrappers `SurvivalBART()` and `SurvivalDART()` provide convenience interfaces for classical BART and DART under the AFT formulation. They calibrate prior hyperparameters automatically from the data range.

```r
set.seed(4)

# SurvivalBART: classical BART prior, AFT likelihood
fit_sbart <- SurvivalBART(
  time            = y_syn,
  status          = d_syn,
  X_train         = X_syn,
  number_of_trees = 5,
  k               = 2.0,
  N_post          = 50,
  N_burn          = 25,
  verbose         = FALSE
)

# SurvivalDART: Dirichlet (DART) splitting prior
fit_sdart <- SurvivalDART(
  time            = y_syn,
  status          = d_syn,
  X_train         = X_syn,
  number_of_trees = 5,
  k               = 2.0,
  N_post          = 50,
  N_burn          = 25,
  verbose         = FALSE
```

```r
)

cat("SurvivalBART  class:", class(fit_sbart), "\n")
#> SurvivalBART  class: ShrinkageTrees
cat("SurvivalDART  class:", class(fit_sdart), "\n")
#> SurvivalDART  class: ShrinkageTrees
```

0.4 High-Dimensional Survival Analysis

A key motivation for horseshoe shrinkage and the Dirichlet (DART) sparsity prior is their behaviour in the $p \gg n$ **regime**: many covariates are available but only a small subset drives the outcome. Classical BART may struggle here because the standard Gaussian leaf prior is non-sparse and does not concentrate on a small number of predictors.

We illustrate both priors on a sparse AFT simulation: $n = 60$ observations, $p = 200$ predictors, and only three active predictors.

```r
set.seed(20)
n_hd <- 60;  p_hd <- 200
X_hd <- matrix(rnorm(n_hd * p_hd), n_hd, p_hd)

# True log-survival depends only on predictors 1, 2, and 3
log_T_hd <- 1.5 * X_hd[, 1] - 1.0 * X_hd[, 2] + 0.5 * X_hd[, 3] + rnorm(n_hd)
C_hd     <- rexp(n_hd, rate = 0.5)
y_hd     <- pmin(exp(log_T_hd), C_hd)
d_hd     <- as.integer(exp(log_T_hd) <= C_hd)

cat("n =", n_hd, "| p =", p_hd,
    "| active predictors = 3",
    "| censoring rate =", round(1 - mean(d_hd), 2), "\n")
#> n = 60 | p = 200 | active predictors = 3 | censoring rate = 0.4
```

ShrinkageTrees (horseshoe) places global–local shrinkage on the step heights of every leaf, automatically regularising all 200 predictors toward zero while preserving the signal in the three active ones.

```r
set.seed(21)
fit_hd_hs <- ShrinkageTrees(
  y               = y_hd,
  status          = d_hd,
  X_train         = X_hd,
  outcome_type    = "right-censored",
  prior_type      = "horseshoe",
  local_hp        = 0.1 / sqrt(10),
  global_hp       = 0.1 / sqrt(10),
  number_of_trees = 10,
  N_post          = 50,
  N_burn          = 25,
  verbose         = FALSE
)
```

SurvivalDART uses a Dirichlet prior on split probabilities to induce structural sparsity: after burn-in, most splitting probability is concentrated on truly predictive variables. Setting `rho_dirichlet = 3` encodes the prior belief that approximately three predictors are active.

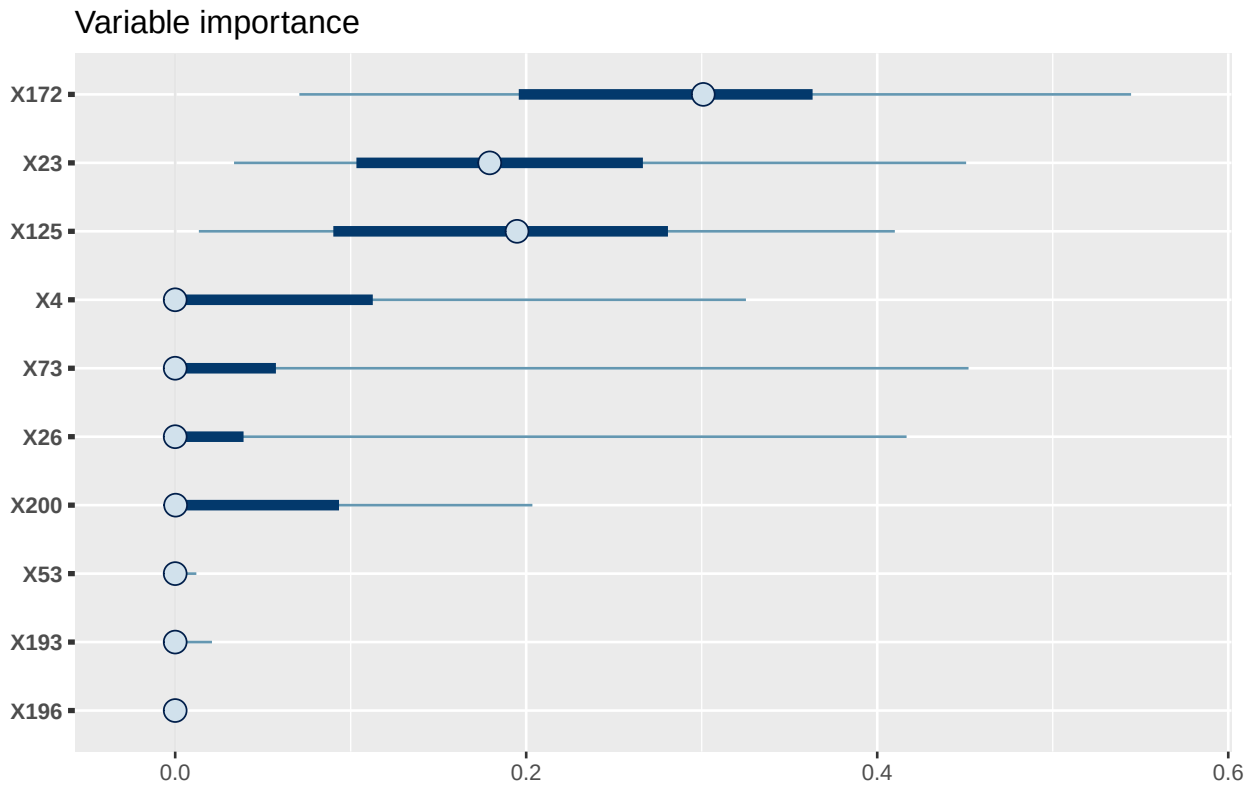
```
set.seed(22)
fit_hd_dart <- SurvivalDART(
  time            = y_hd,
  status          = d_hd,
  X_train         = X_hd,
  number_of_trees = 10,
  rho_dirichlet   = 3,
  N_post          = 50,
  N_burn          = 25,
  verbose         = FALSE
)
```

Both models run without error in the $p > n$ regime. We compare their posterior mean predictions in log-time against the latent true values used to generate the data.

```
rmse_hs   <- sqrt(mean((fit_hd_hs$train_predictions  - log_T_hd)^2))
rmse_dart <- sqrt(mean((fit_hd_dart$train_predictions - log_T_hd)^2))

cat(sprintf("%-18s  train RMSE (log-time): %.3f\n", "Horseshoe", rmse_hs))
#> Horseshoe           train RMSE (log-time): 2.530
cat(sprintf("%-18s  train RMSE (log-time): %.3f\n", "DART",      rmse_dart))
#> DART                train RMSE (log-time): 2.402
```

The DART model also produces **variable importance** plots that display the posterior distribution of each predictor's splitting probability. With only 50 posterior draws the top-10 plot below should already concentrate most probability mass near the three truly active predictors.

```
plot(fit_hd_dart, type = "vi", n_vi = 10)
```



0.5 Causal Forest Models

The causal models decompose the outcome as

$$Y_i = \mu(\mathbf{x}_i) + W_i \cdot \tau(\mathbf{x}_i) + \varepsilon_i,$$

where $\mu(\cdot)$ is the prognostic (control) function and $\tau(\cdot)$ is the heterogeneous treatment effect. Separate tree ensembles are fitted for each component.

We generate synthetic data with a known treatment effect for illustration.

```
set.seed(5)
tau_true <- 1.5 * (X_syn[, 1] > 0)     # heterogeneous treatment effect
y_causal <- X_syn[, 1] + W_syn * tau_true + rnorm(n, sd = 0.5)
```

0.5.1 SurvivalBCF — classical BCF for survival

`SurvivalBCF()` fits a BCF model for right-censored survival outcomes using classical BART priors.

```
# Full analysis (eval=FALSE - use larger MCMC settings in practice)
fit_sbcf <- SurvivalBCF(
  time       = time,
```

```r
  status     = status,
  X_train    = X,
  treatment  = treatment,
  propensity = propensity,   # from HorseTrees above
  N_post     = 5000,
  N_burn     = 5000,
  verbose    = FALSE
)
```

```r
set.seed(6)
fit_sbcf <- SurvivalBCF(
  time            = y_syn,
  status          = d_syn,
  X_train         = X_syn,
  treatment       = W_syn,
  number_of_trees_control = 5,
  number_of_trees_treat   = 5,
  N_post          = 50,
  N_burn          = 25,
  verbose         = FALSE
)
cat("SurvivalBCF class:", class(fit_sbcf), "\n")
#> SurvivalBCF class: CausalShrinkageForest
cat("ATE (posterior mean):",
    round(mean(fit_sbcf$train_predictions_treat), 3), "\n")
#> ATE (posterior mean): 1.475
```

0.5.2 SurvivalShrinkageBCF — sparse causal survival forest

`SurvivalShrinkageBCF()` extends BCF with a Dirichlet splitting prior on both forests, inducing sparsity in high-dimensional settings.

```r
set.seed(7)
fit_ssbcf <- SurvivalShrinkageBCF(
  time            = y_syn,
  status          = d_syn,
  X_train         = X_syn,
  treatment       = W_syn,
  number_of_trees_control = 5,
  number_of_trees_treat   = 5,
  N_post          = 50,
  N_burn          = 25,
  verbose         = FALSE
)
cat("SurvivalShrinkageBCF class:", class(fit_ssbcf), "\n")
#> SurvivalShrinkageBCF class: CausalShrinkageForest
```

0.5.3 CausalHorseForest — horseshoe causal forest

`CausalHorseForest()` is the primary novel contribution of this package. It applies horseshoe shrinkage to the leaf parameters of both the prognostic and treatment-effect forests. This enables effective regularisation when many covariates are available but few are truly predictive of heterogeneous treatment effects.

```
set.seed(8)
fit_chf <- CausalHorseForest(
  y                        = y_causal,
  X_train_control          = X_syn,
  X_train_treat            = X_syn,
  treatment_indicator_train = W_syn,
  outcome_type             = "continuous",
  number_of_trees          = 5,
  N_post                   = 50,
  N_burn                   = 25,
  store_posterior_sample   = TRUE,
  verbose                  = FALSE
)

cat("CausalHorseForest class:", class(fit_chf), "\n")
#> CausalHorseForest class: CausalShrinkageForest

# Posterior mean CATE
cate_mean <- fit_chf$train_predictions_treat
cat("CATE - posterior mean (first 5):",
    round(cate_mean[1:5], 3), "\n")
#> CATE - posterior mean (first 5): 1.695 1.544 1.568 1.66 1.642

# Posterior ATE
ate_samples <- rowMeans(fit_chf$train_predictions_sample_treat)
cat("ATE posterior mean:",
    round(mean(ate_samples), 3),
    "  95% CI: [",
    round(quantile(ate_samples, 0.025), 3), ",",
    round(quantile(ate_samples, 0.975), 3), "]\n")
#> ATE posterior mean: 1.619   95% CI: [ 1.191 , 1.926 ]
```

You can also supply separate test matrices to obtain out-of-sample predictions.

```
set.seed(9)
X_test <- matrix(rnorm(20 * p), 20, p)
W_test <- rbinom(20, 1, 0.5)

fit_chf_test <- CausalHorseForest(
  y                        = y_causal,
  X_train_control          = X_syn,
  X_train_treat            = X_syn,
  treatment_indicator_train = W_syn,
  X_test_control           = X_test,
  X_test_treat             = X_test,
  treatment_indicator_test  = W_test,
  outcome_type             = "continuous",
  number_of_trees          = 5,
```

```r
  N_post                    = 50,
  N_burn                    = 25,
  store_posterior_sample    = TRUE,
  verbose                   = FALSE
)

cat("Test CATE (first 5):",
    round(fit_chf_test$test_predictions_treat[1:5], 3), "\n")
#> Test CATE (first 5): 1.645 1.599 1.707 1.651 1.621
```

0.5.4 CausalShrinkageForest — flexible causal priors

`CausalShrinkageForest()` is the most general interface. It allows independent prior choices for the control and treatment forests via `prior_type_control` and `prior_type_treat`.

```r
set.seed(10)
lh <- 0.1 / sqrt(5)

fit_csf <- CausalShrinkageForest(
  y                         = y_causal,
  X_train_control           = X_syn,
  X_train_treat             = X_syn,
  treatment_indicator_train = W_syn,
  outcome_type              = "continuous",
  prior_type_control        = "horseshoe",
  prior_type_treat          = "horseshoe",
  local_hp_control          = lh,
  global_hp_control         = lh,
  local_hp_treat            = lh,
  global_hp_treat           = lh,
  number_of_trees_control   = 5,
  number_of_trees_treat     = 5,
  N_post                    = 50,
  N_burn                    = 25,
  store_posterior_sample    = TRUE,
  verbose                   = FALSE
)

cat("CausalShrinkageForest class:", class(fit_csf), "\n")
#> CausalShrinkageForest class: CausalShrinkageForest
cat("Acceptance ratio (control):",
    round(fit_csf$acceptance_ratio_control, 3), "\n")
#> Acceptance ratio (control): 0.324
cat("Acceptance ratio (treat)  :",
    round(fit_csf$acceptance_ratio_treat, 3), "\n")
#> Acceptance ratio (treat)  : 0.428
```

The `horseshoe_fw` prior adds a forest-wide shrinkage parameter that is tracked in the fitted object.

```r
set.seed(11)
fit_fw2 <- CausalShrinkageForest(
  y                         = y_causal,
  X_train_control           = X_syn,
  X_train_treat             = X_syn,
  treatment_indicator_train = W_syn,
  outcome_type              = "continuous",
  prior_type_control        = "horseshoe_fw",
  prior_type_treat          = "horseshoe_fw",
  local_hp_control          = lh,
  global_hp_control         = lh,
  local_hp_treat            = lh,
  global_hp_treat           = lh,
  number_of_trees_control   = 5,
  number_of_trees_treat     = 5,
  N_post                    = 50,
  N_burn                    = 25,
  verbose                   = FALSE
)

cat("Forest-wide shrinkage (control, first 5 draws):\n")
#> Forest-wide shrinkage (control, first 5 draws):
print(round(fit_fw2$forestwide_shrinkage_control[1:5], 4))
#> [1] 1 1 1 1 1
```

0.6 S3 Methods

All fitted objects support `print()`, `summary()`, `predict()`, and `plot()`.

0.6.1 print()

Calling `print()` (or just typing the object name) displays a concise model summary.

```r
print(fit_chf)
#>
#> CausalShrinkageForest model
#> ---------------------------
#> Outcome type:         Continuous
#> Training size (n):    80
#> Posterior draws:      50 (burn-in 25)
#> Posterior mean sigma: 0.814
#>
#>                       Control             Treatment
#>                       -------------------- --------------------
#> Prior:                horseshoe           horseshoe
#> Number of trees:      5                   5
#> Number of features:   10                  10
#> Acceptance ratio:     0.436               0.476
```

For causal models the output additionally shows the number of trees in each forest and prior details for both components.

```
print(fit_csf)
#>
#> CausalShrinkageForest model
#> ---------------------------
#> Outcome type:         Continuous
#> Training size (n):    80
#> Posterior draws:      50 (burn-in 25)
#> Posterior mean sigma: 0.812
#>
#>                       Control             Treatment
#>                       -------------------- --------------------
#> Prior:                horseshoe           horseshoe
#> Number of trees:      5                   5
#> Number of features:   10                  10
#> Acceptance ratio:     0.324               0.428
```

0.6.2 summary()

`summary()` returns a structured list and displays a richer description including posterior statistics for σ, acceptance ratios, and treatment effect estimates.

```
smry <- summary(fit_hs)
print(smry)
#>
#> ShrinkageTrees model summary
#> ============================
#> Call: ShrinkageTrees(y = y_cont, X_train = X_syn, outcome_type = "continuous",
#>     number_of_trees = 5, prior_type = "horseshoe", local_hp = 0.1/sqrt(5),
#>     global_hp = 0.1/sqrt(5), N_post = 50, N_burn = 25, verbose = FALSE)
#>
#> Outcome: Continuous | Prior: horseshoe | Trees: 5
#> Data:    n = 80, p = 10 | Draws: 50 (burn-in 25)
#>
#> Posterior sigma:
#>   Mean: 0.987  SD: 0.089  95% CI: [0.851, 1.205]
#>
#> Predictions (posterior mean):
#>   Train: mean = 0.053, sd = 0.053, range = [-0.094, 0.159]
#>   Test:  mean = 0.065, sd = NA, range = [0.065, 0.065]
#>
#> Variable importance (posterior inclusion probability):
#>   X3: 0.164   X9: 0.143   X4: 0.142   X2: 0.132   X8: 0.131   X5: 0.091   X7: 0.084   X6: 0.064   X1
#>
#> MCMC acceptance ratio: 0.484
```

For causal models the summary includes the posterior ATE with a 95% credible interval (when `store_posterior_sample = TRUE`).

```r
smry_c <- summary(fit_chf)
print(smry_c)
#>
#> CausalShrinkageForest model summary
#> ====================================
#> Call: CausalHorseForest(y = y_causal, X_train_control = X_syn, X_train_treat = X_syn,
#>     treatment_indicator_train = W_syn, outcome_type = "continuous",
#>     number_of_trees = 5, N_post = 50, N_burn = 25, store_posterior_sample = TRUE,
#>     verbose = FALSE)
#>
#> Outcome: Continuous
#> Prior:   control = horseshoe, treatment = horseshoe
#> Trees:   control = 5, treatment = 5
#> Data:    n = 80, p_control = 10, p_treat = 10 | Draws: 50 (burn-in 25)
#>
#> Treatment effect (CATE):
#>   ATE:     1.6186  95% CI: [1.1907, 1.9258]
#>   CATE SD: 0.0755
#>
#> Prognostic function (mu):
#>   Mean: 0.588  SD: 0.016  Range: [0.554, 0.621]
#>
#> Posterior sigma:
#>   Mean: 0.814  SD: 0.063  95% CI: [0.699, 0.942]
#>
#> Variable importance - control forest (posterior inclusion probability):
#>   X3: 0.183   X4: 0.182   X8: 0.143   X2: 0.142   X6: 0.099   X9: 0.066   X10: 0.061   X5: 0.048   X
#>
#> Variable importance - treatment forest (posterior inclusion probability):
#>   X5: 0.242   X7: 0.144   X6: 0.115   X8: 0.1   X9: 0.086   X1: 0.082   X3: 0.079   X2: 0.067   X10:
#>
#> MCMC acceptance ratios: control = 0.436, treatment = 0.476

# Access the ATE directly
cat("ATE mean  :", round(smry_c$treatment_effect$ate, 3), "\n")
#> ATE mean  : 1.619
cat("ATE 95% CI: [",
    round(smry_c$treatment_effect$ate_lower, 3), ",",
    round(smry_c$treatment_effect$ate_upper, 3), "]\n")
#> ATE 95% CI: [ 1.191 , 1.926 ]
```

0.6.3 predict()

`predict()` computes the posterior predictive distribution on new data. It returns a `ShrinkageTreesPrediction` object with posterior mean and credible-interval vectors.

```r
X_new  <- matrix(rnorm(10 * p), 10, p)

pred <- predict(fit_hs, newdata = X_new)
print(pred)
```

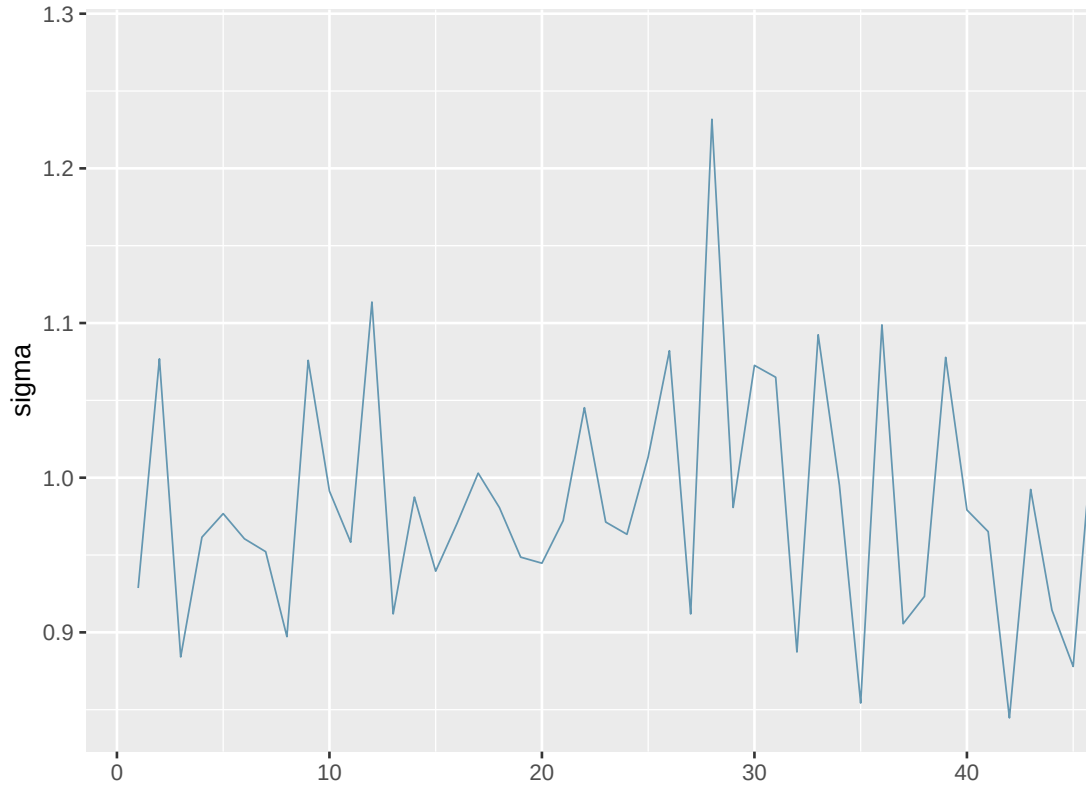
```
#>
#> ShrinkageTrees predictions
#> --------------------------
#> Observations:         10
#> Credible interval:    95%
#> Scale:                fitted value
#>
#>              mean     lower     upper
#>          --------  --------  --------
#>   [  1]     0.052    -0.192     0.300
#>   [  2]     0.026    -0.275     0.339
#>   [  3]     0.010    -0.312     0.309
#>   [  4]     0.042    -0.264     0.299
#>   [  5]     0.066    -0.271     0.663
#>   [  6]     0.058    -0.162     0.378
#>   ... (4 more)
```

```
# Point estimates and 95% credible intervals
head(data.frame(
  mean  = round(pred$mean,  3),
  lower = round(pred$lower, 3),
  upper = round(pred$upper, 3)
))
#>    mean  lower upper
#> 1 0.052 -0.192 0.300
#> 2 0.026 -0.275 0.339
#> 3 0.010 -0.312 0.309
#> 4 0.042 -0.264 0.299
#> 5 0.066 -0.271 0.663
#> 6 0.058 -0.162 0.378
```

0.6.4 plot()

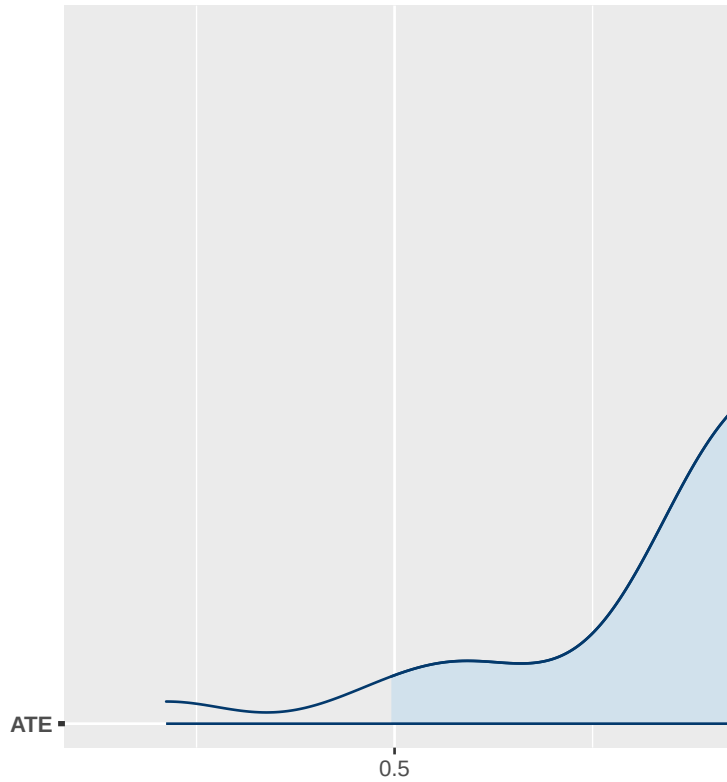
The `plot()` method produces diagnostic and inferential graphics using the **bayesplot** and **ggplot2** packages (both are suggested dependencies).

```
plot(fit_hs, type = "trace")
```



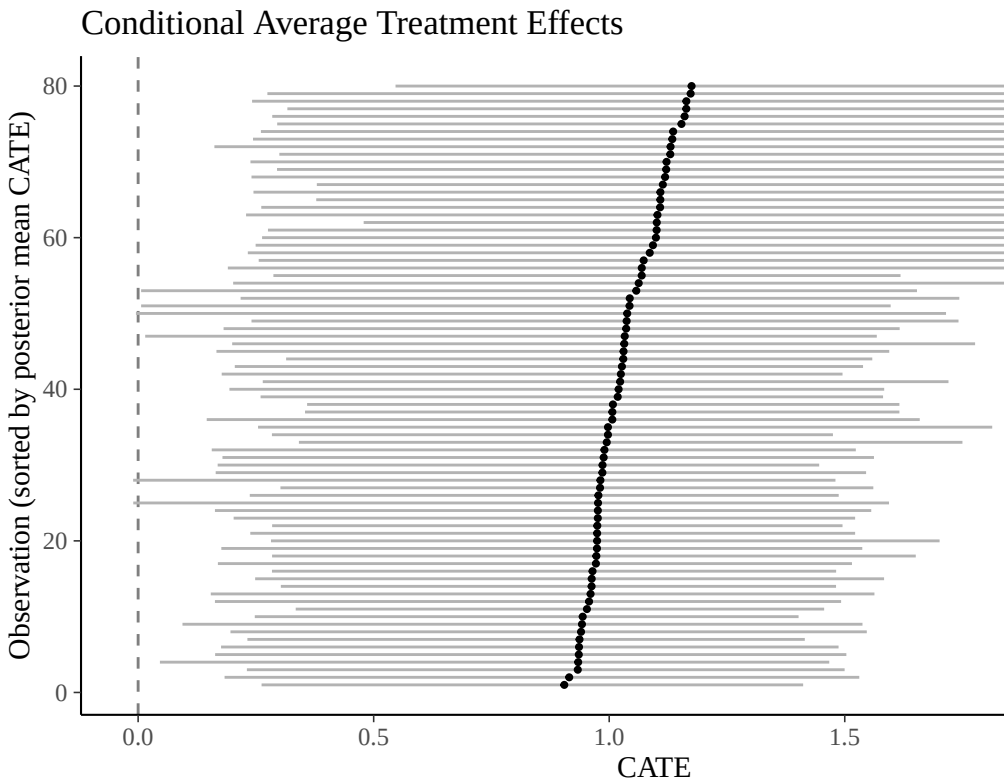
0.6.4.1 Sigma traceplot

```
plot(fit_chf, type = "ate")
```



0.6.4.2 Posterior ATE distribution (causal models)

```
plot(fit_chf, type = "cate")
```



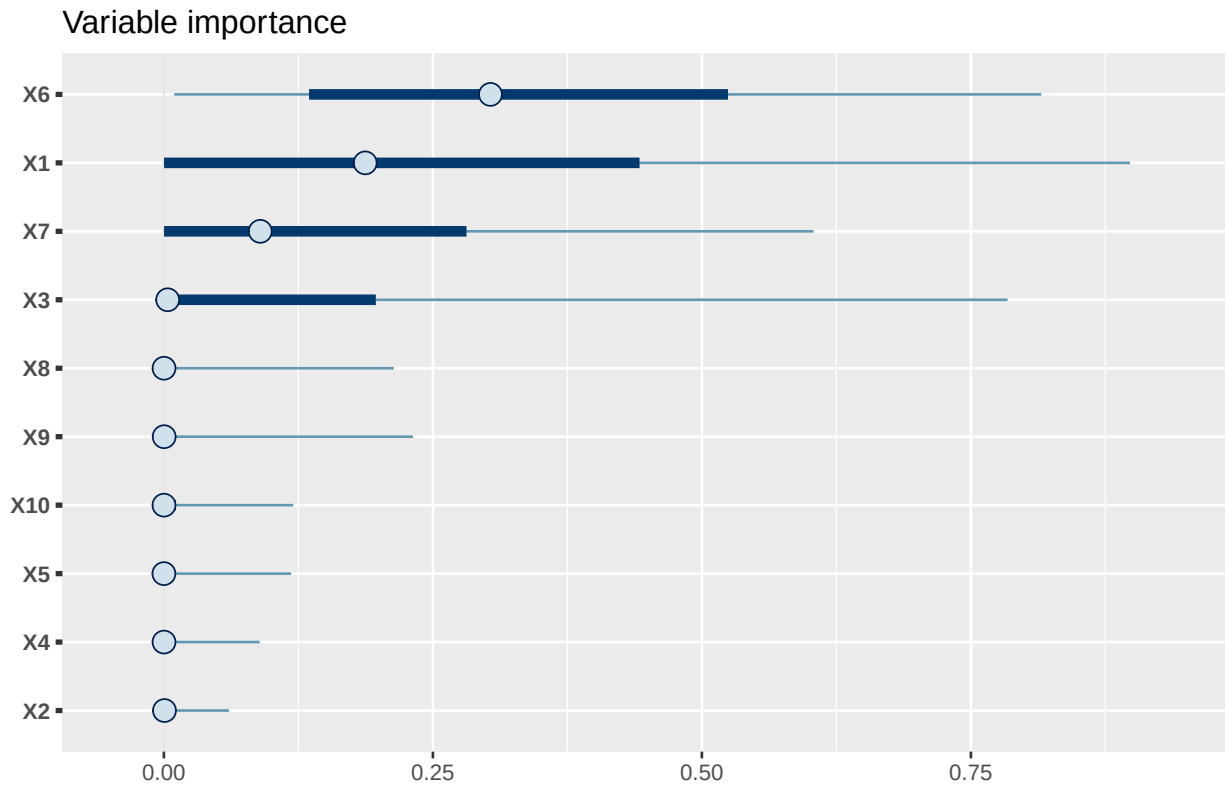
0.6.4.3 CATE caterpillar plot

0.6.4.4 Variable importance (Dirichlet prior)

Variable importance plots are available when `prior_type = "dirichlet"`. For causal models with `prior_type_control = "dirichlet"` or `prior_type_treat = "dirichlet"`, use `forest = "control"`, `"treat"`, or `"both"`.

```
set.seed(12)
fit_dart <- ShrinkageTrees(
  y               = y_cont,
  X_train         = X_syn,
  outcome_type    = "continuous",
  prior_type      = "dirichlet",
  local_hp        = 0.1 / sqrt(5),
  number_of_trees = 5,
  N_post          = 50,
  N_burn          = 25,
  verbose         = FALSE
)
```

```
plot(fit_dart, type = "vi", n_vi = 10)
```



0.7 Multi-Chain MCMC

Running multiple independent chains improves mixing diagnostics and reduces sensitivity to starting values. Pass `n_chains > 1` to any model-fitting function; chains are run in parallel via `parallel::mclapply` on Unix-like systems.

```
set.seed(13)
fit_2chain <- ShrinkageTrees(
  y               = y_cont,
  X_train         = X_syn,
  outcome_type    = "continuous",
  prior_type      = "horseshoe",
  local_hp        = 0.1 / sqrt(5),
  global_hp       = 0.1 / sqrt(5),
  number_of_trees = 5,
  N_post          = 50,
  N_burn          = 25,
  n_chains        = 2,
  verbose         = FALSE
)

cat("n_chains stored  :", fit_2chain$mcmc$n_chains, "\n")
#> n_chains stored  : 2
cat("Total sigma draws:", length(fit_2chain$sigma),
```

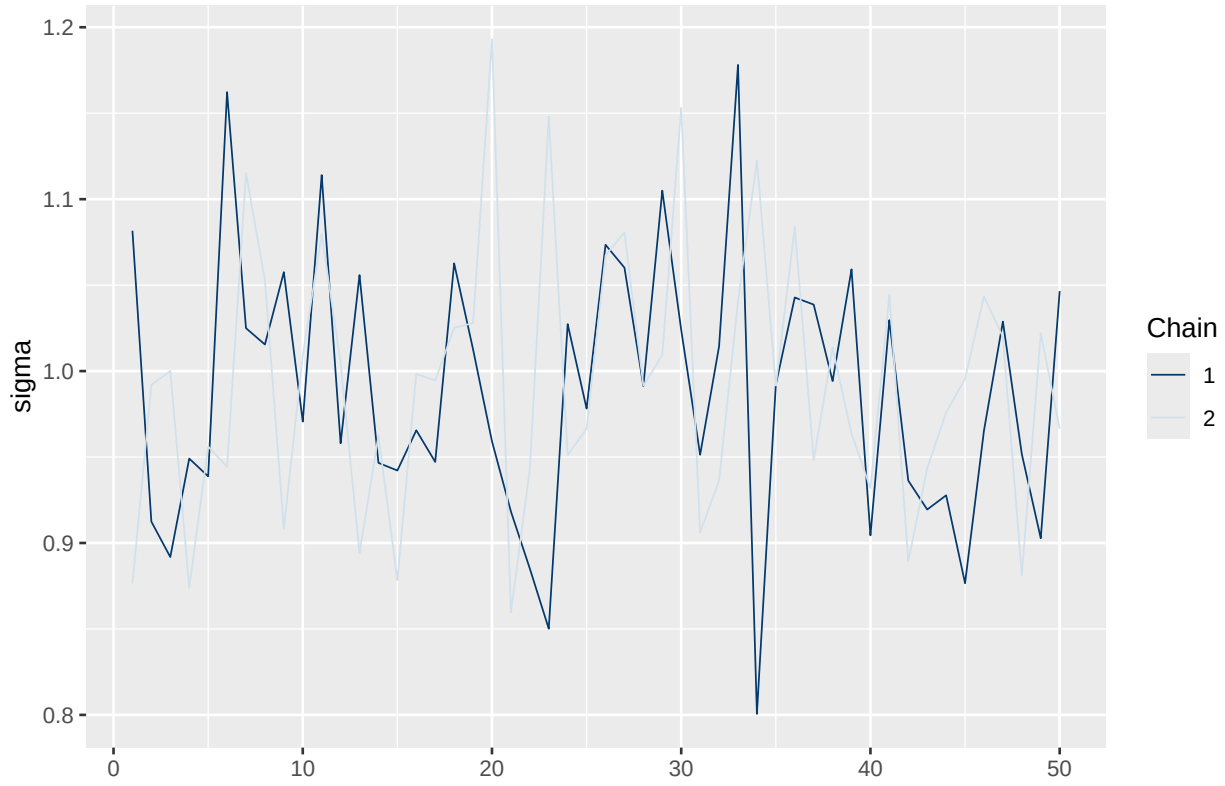
```r
    " (2 chains x 50 draws)\n")
#> Total sigma draws: 100  (2 chains x 50 draws)
cat("Per-chain acceptance ratios:\n")
#> Per-chain acceptance ratios:
print(round(fit_2chain$chains$acceptance_ratios, 3))
#> [1] 0.492 0.504
```

The same interface works for causal models.

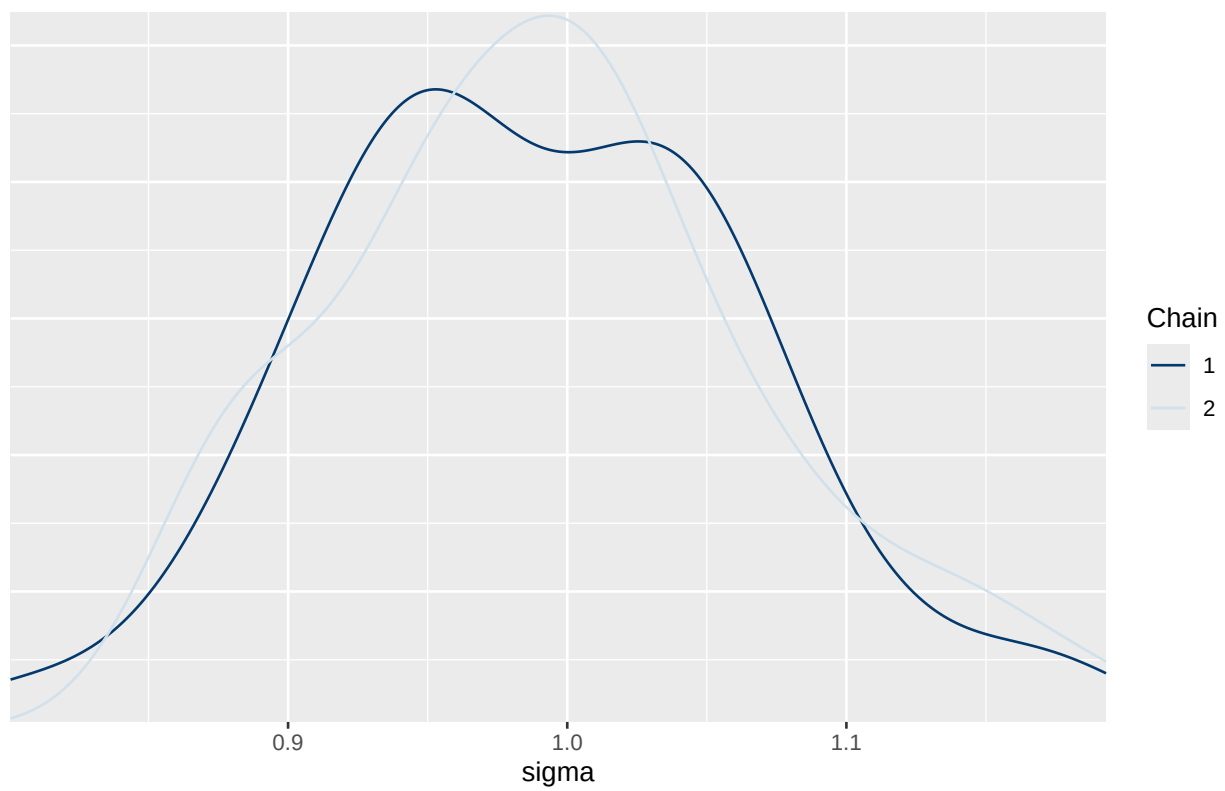
```r
set.seed(14)
fit_causal_2chain <- CausalShrinkageForest(
  y                        = y_causal,
  X_train_control          = X_syn,
  X_train_treat            = X_syn,
  treatment_indicator_train = W_syn,
  outcome_type             = "continuous",
  prior_type_control       = "horseshoe",
  prior_type_treat         = "horseshoe",
  local_hp_control         = lh,
  global_hp_control        = lh,
  local_hp_treat           = lh,
  global_hp_treat          = lh,
  number_of_trees_control  = 5,
  number_of_trees_treat    = 5,
  N_post                   = 50,
  N_burn                   = 25,
  n_chains                 = 2,
  verbose                  = FALSE
)

cat("Pooled sigma draws:", length(fit_causal_2chain$sigma), "\n")
#> Pooled sigma draws: 100
cat("Per-chain acceptance ratios (control):\n")
#> Per-chain acceptance ratios (control):
print(round(fit_causal_2chain$chains$acceptance_ratios_control, 3))
#> [1] 0.320 0.356
```

With multiple chains the traceplot shows one line per chain, and the overlaid density plot compares the marginal posterior of σ across chains — both require `n_chains > 1`.

```r
plot(fit_2chain, type = "trace")
```




```
plot(fit_2chain, type = "density")
```



0.8 Case Study: TCGA PAAD (Full Analysis)

The full analysis of the `pdac` dataset replicates the case study from Jacobs, van Wieringen & van der Pas (2025). Due to the high-dimensional covariate space (~3,000 genes) and the large MCMC settings needed for reliable inference, the code below is provided for reference but is not evaluated during vignette building. Pre-computed results can be reproduced by running the `pdac_analysis` demo: `demo("pdac_analysis", package = "ShrinkageTrees")`.

0.8.1 Step 1: Propensity score estimation

```r
data("pdac")

time      <- pdac$time
status    <- pdac$status
treatment <- pdac$treatment
X         <- as.matrix(pdac[, !(names(pdac) %in% c("time","status","treatment"))])

set.seed(2025)
ps_fit <- HorseTrees(
  y            = treatment,
  X_train      = X,
  outcome_type = "binary",
  k            = 0.1,
  N_post       = 5000,
  N_burn       = 5000,
  verbose      = FALSE
)

propensity <- pnorm(ps_fit$train_predictions)
```

```r
# Overlap plot
p0 <- propensity[treatment == 0]
p1 <- propensity[treatment == 1]

hist(p0, breaks = 15, col = rgb(1, 0.5, 0, 0.5), xlim = range(propensity),
     xlab = "Propensity score", main = "Propensity score overlap")
hist(p1, breaks = 15, col = rgb(0, 0.5, 0, 0.5), add = TRUE)
legend("topright", legend = c("Control", "Treated"),
       fill = c(rgb(1,0.5,0,0.5), rgb(0,0.5,0,0.5)))
```

0.8.2 Step 2: Causal survival forest

```r
# Augment control matrix with propensity scores (BCF-style)
X_control <- cbind(propensity, X)

# Log-transform and centre survival times
log_time <- log(time) - mean(log(time))
```

```r
set.seed(2025)
fit_pdac <- CausalHorseForest(
  y                          = log_time,
  status                     = status,
  X_train_control            = X_control,
  X_train_treat              = X,
  treatment_indicator_train = treatment,
  outcome_type               = "right-censored",
  timescale                  = "log",
  number_of_trees            = 200,
  N_post                     = 5000,
  N_burn                     = 5000,
  store_posterior_sample     = TRUE,
  verbose                    = FALSE
)
```

0.8.3 Step 3: ATE and CATE estimation

```r
# Print model summary
print(fit_pdac)
smry_pdac <- summary(fit_pdac)
print(smry_pdac)

# ATE
cat("ATE posterior mean:",
    round(smry_pdac$treatment_effect$ate, 3), "\n")
cat("95% CI: [",
    round(smry_pdac$treatment_effect$ate_lower, 3), ",",
    round(smry_pdac$treatment_effect$ate_upper, 3), "]\n")
```

0.8.4 Step 4: Diagnostics

```r
# Sigma convergence
plot(fit_pdac, type = "trace")

# Posterior ATE density
plot(fit_pdac, type = "ate")

# CATE caterpillar plot
plot(fit_pdac, type = "cate")
```

0.9 Session Info

```r
sessionInfo()
#> R version 4.4.2 (2024-10-31)
```

```
#> Platform: aarch64-apple-darwin20
#> Running under: macOS 26.3
#>
#> Matrix products: default
#> BLAS:   /Library/Frameworks/R.framework/Versions/4.4-arm64/Resources/lib/libRblas.0.dylib
#> LAPACK: /Library/Frameworks/R.framework/Versions/4.4-arm64/Resources/lib/libRlapack.dylib;  LAPACK v
#>
#> locale:
#> [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
#>
#> time zone: Europe/Berlin
#> tzcode source: internal
#>
#> attached base packages:
#> [1] stats     graphics  grDevices utils     datasets  methods   base
#>
#> other attached packages:
#> [1] ShrinkageTrees_1.3.0
#>
#> loaded via a namespace (and not attached):
#>  [1] Matrix_1.7-3         bayesplot_1.13.0     gtable_0.3.6
#>  [4] dplyr_1.2.0          compiler_4.4.2       tidyselect_1.2.1
#>  [7] Rcpp_1.1.1           stringr_1.6.0        parallel_4.4.2
#> [10] splines_4.4.2        scales_1.4.0         yaml_2.3.12
#> [13] fastmap_1.2.0        lattice_0.22-7       plyr_1.8.9
#> [16] ggplot2_4.0.0        R6_2.6.1             labeling_0.4.3
#> [19] generics_0.1.4       distributional_0.5.0 knitr_1.50
#> [22] backports_1.5.0      checkmate_2.3.2      tibble_3.3.1
#> [25] pillar_1.11.1        RColorBrewer_1.1-3   posterior_1.6.1
#> [28] rlang_1.1.7          stringi_1.8.7        xfun_0.52
#> [31] S7_0.2.0             cli_3.6.5            withr_3.0.2
#> [34] magrittr_2.0.4       digest_0.6.37        grid_4.4.2
#> [37] rstudioapi_0.18.0    lifecycle_1.0.5      vctrs_0.7.1
#> [40] evaluate_1.0.4       glue_1.8.0           tensorA_0.36.2.1
#> [43] farver_2.1.2         survival_3.8-3       abind_1.4-8
#> [46] reshape2_1.4.4       rmarkdown_2.29       ggridges_0.5.6
#> [49] tools_4.4.2          pkgconfig_2.0.3      htmltools_0.5.8.1
```

0.10 References

Chipman, H. A., George, E. I., & McCulloch, R. E. (2010). Bayesian Additive Regression Trees. *Annals of Applied Statistics*, 4(1), 266–298.

Hahn, P. R., Murray, J. S., & Carvalho, C. M. (2020). Bayesian Regression Tree Models for Causal Inference: Regularization, Confounding, and Heterogeneous Treatment Effects. *Bayesian Analysis*, 15(3), 965–1056.

Jacobs, T., van Wieringen, W. N., & van der Pas, S. L. (2025). Horseshoe Forests for High-Dimensional Causal Survival Analysis. *arXiv preprint* arXiv:2507.22004.

Linero, A. R. (2018). Bayesian Regression Trees for High-Dimensional Prediction and Variable Selection. *Journal of the American Statistical Association*, 113(522), 626–636.

Sparapani, R., Spanbauer, C., & McCulloch, R. (2021). Nonparametric Machine Learning and Efficient Computation with Bayesian Additive Regression Trees: The BART R Package. *Journal of Statistical Software*, 97(1), 1–66.